

Experiment-6: Exploring Inheritance types in Object-Oriented Programming

Objective: The objective of this experiment is to introduce students to the concept of inheritance in object-oriented programming, enabling them to understand how one class can inherit properties and behaviors from another. Through simple Java implementations of single and multilevel inheritance, learners will observe how code reusability, method overriding, and the use of the super keyword contribute to cleaner, modular, and maintainable software design.

Inheritance: Inheritance is a fundamental concept in object-oriented programming (OOP) that allows one class (called the subclass or child class) to acquire the properties and behaviors (fields and methods) of another class (called the superclass or parent class). It promotes code reusability, modularity, and extensibility, enabling developers to build upon existing code structures without rewriting them.

(Dear students refer 24th class document discussed on 8-SEPT-2026 for detailed explanation)

Inheritance classification:

Single Inheritance: One parent class, one child class.

Syntax:

```
class Parent {  
    // parent properties & methods  
}
```

```
class Child extends Parent {  
    // child properties & methods  
}
```

Example:

```
class Animal {  
    void eat() {  
        System.out.println("Animal is eating");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog is barking");  
    }  
}
```

```
public class SingleInheritanceDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat(); // from parent  
        d.bark(); // from child  
    }  
}
```

Output:

Animal is eating
Dog is barking

2. Multilevel Inheritance: A class inherits from another class, which in turn is inherited by another class.

Syntax:

```
class A { }  
class B extends A { }  
class C extends B { }
```

Example:

```

class Animal {
    void eat() { System.out.println("Eating..."); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}

class Puppy extends Dog {
    void weep() { System.out.println("Weeping..."); }
}

public class MultilevelInheritanceDemo {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
        p.weep();
    }
}

```

Output:

Eating...
Barking...
Weeping...

3. Hierarchical Inheritance: Multiple classes inherit from the same parent.

Syntax:

```

class Parent { }
class Child1 extends Parent { }
class Child2 extends Parent { }

```

Example:

```

class Animal {
    void eat() { System.out.println("Eating..."); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}

class Cat extends Animal {
    void meow() { System.out.println("Meowing..."); }
}

public class HierarchicalInheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();

        Cat c = new Cat();
        c.eat();
        c.meow();
    }
}

```

Output:

Eating...
Barking...

Eating...
Meowing...

4. Hybrid Inheritance (via Interfaces in Java): Combination of multiple inheritance and other types. (Java does not allow multiple inheritance with classes, but we use interfaces to achieve hybrid.)

Syntax:

```
class Parent { }  
interface Interface1 { }  
interface Interface2 { }  
class Child extends Parent implements Interface1, Interface2 { }
```

Example:

```
interface CanRun {  
    void run();  
}  
  
interface CanSwim {  
    void swim();  
}  
  
class Animal {  
    void eat() { System.out.println("Eating..."); }  
}  
  
class Amphibian extends Animal implements CanRun, CanSwim {  
    public void run() { System.out.println("Running..."); }  
    public void swim() { System.out.println("Swimming..."); }  
}  
  
public class HybridInheritanceDemo {  
    public static void main(String[] args) {  
        Amphibian a = new Amphibian();  
        a.eat();  
        a.run();  
        a.swim();  
    }  
}
```

Output:

Eating...
Running...
Swimming...

5. Multiple Inheritance (through Interfaces): A class implements more than one interface.

Syntax:

```
interface Interface1 { }  
interface Interface2 { }  
class Child implements Interface1, Interface2 { }
```

Example:

```
interface Printer {  
    void print();  
}  
  
interface Scanner {  
    void scan();  
}  
  
class MultiFunctionDevice implements Printer, Scanner {  
    public void print() { System.out.println("Printing document..."); }  
    public void scan() { System.out.println("Scanning document..."); }  
}
```

```

}

public class MultipleInheritanceDemo {
    public static void main(String[] args) {
        MultiFunctionDevice mfd = new MultiFunctionDevice();
        mfd.print();
        mfd.scan();
    }
}

```

Output:

Printing document...
Scanning document...

Type	Description
Single Inheritance	A subclass inherits from one superclass.
Multilevel Inheritance	A subclass inherits from a superclass, and another subclass inherits from that subclass.
Hierarchical Inheritance	Multiple subclasses inherit from a single superclass.
Multiple Inheritance (via interfaces)	A class inherits from multiple interfaces (Java doesn't support multiple class inheritance).
Hybrid Inheritance	A combination of two or more types of inheritance.

Benefits of Inheritance in OOP

- ✓ **Code Reusability:** Inheritance allows child classes to reuse methods and properties from parent classes, reducing redundancy and promoting the DRY (Don't Repeat Yourself) principle.
- ✓ **Logical Class Hierarchy:** It helps organize classes into a clear hierarchy, making complex systems easier to understand and maintain.
- ✓ **Extensibility:** New features can be added to existing classes by extending them, without altering the original codebase.
- ✓ **Method Overriding and Polymorphism:** Enables runtime polymorphism by allowing subclasses to override parent class methods, enhancing flexibility and dynamic behavior.
- ✓ **Improved Maintainability:** Centralized logic in base classes simplifies updates and bug fixes, as changes propagate to all derived classes.
- ✓ **Abstraction Support:** Inheritance supports abstraction by allowing base classes to define general behavior while subclasses implement specific details.

Students To Address Following Inheritance Based Programming Concepts

1. Single Inheritance

Experiment 6.1: Implementation of Single Inheritance Using a Library Management System

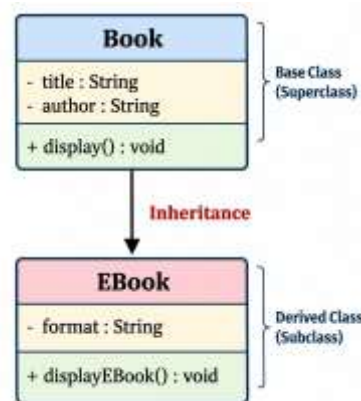
Aim: To design and implement a Library Management System using single inheritance, demonstrating how a derived class can reuse the attributes and methods of a base class and add its own functionality.

Problem Statement

Develop a program for a Library Management System using single inheritance.

1. Create a base class Book with the following data members:
 - o title – title of the book
 - o author – name of the author
2. Implement a suitable constructor and a method displayBookDetails() in the Book class to initialize and display the book information.
3. Create a derived class EBook that inherits from Book and contains an additional data member:
 - o format – electronic format of the book such as PDF, EPUB, etc.
4. Implement a constructor and a method displayEBookDetails() in the EBook class to display the complete details of an e-book.
5. Create **at least four objects** of the EBook class and display their details.
6. Demonstrate that an EBook object can access:
 - o Members inherited from the Book class.
 - o Members defined within the EBook class.

Class Relationship



Relationship: EBook IS-A Book

Procedure

1. Define the Book base class.
2. Declare title and author as data members.
3. Define a constructor to initialize the book details.
4. Implement a method to display the book information.
5. Define the EBook class by inheriting from Book.
6. Add the format data member to EBook.
7. Define the EBook constructor and initialize all required attributes.
8. Implement a method to display the complete e-book details.
9. Create four EBook objects using the given test data.
10. Invoke the appropriate methods and verify the output.

Expected Output

===== E-BOOK DETAILS =====

Title : Java Basics

Author : James Gosling

Format : EPUB

Title : Digital Design Fundamentals
Author : Morris Mano
Format : PDF

Title : Data Structures in C
Author : Reema Thareja
Format : PDF

Title : Object-Oriented Programming
Author : Bjarne Stroustrup
Format : PDF

Experiment6.2: Implementation of Multilevel Inheritance Using a Banking Application

Aim: To design and implement a Banking Application using multilevel inheritance, demonstrating the inheritance of properties and methods through multiple levels of a class hierarchy and the constructor invocation chain.

Problem Statement

Develop a Banking Application using **multilevel inheritance** with the following class hierarchy:

1. Create a base class Account with:
 - accountNumber – stores the account number.
 - balance – stores the current account balance.

Implement a constructor to initialize these attributes and a method displayAccountDetails() to display the account information.

2. Create a derived class SavingsAccount that inherits from Account and adds:
 - interestRate – stores the applicable interest rate.

Implement a constructor to initialize the inherited attributes and interestRate.

3. Create another derived class PremiumSavingsAccount that inherits from SavingsAccount and adds:
 - extraBenefits – stores additional benefits provided to premium customers.

Implement a constructor to initialize all attributes.

4. In PremiumSavingsAccount, implement a method calculateTotalBalance() to calculate the balance after adding the interest.

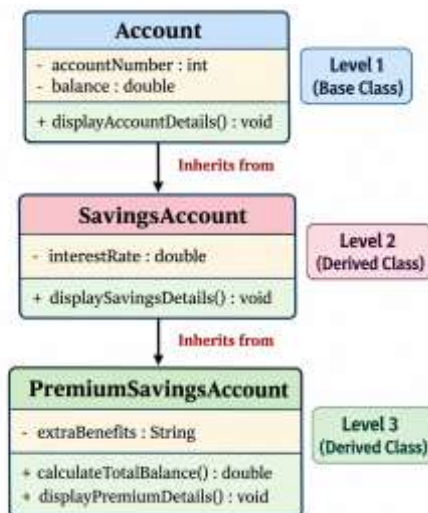
Formula:

$$\text{Interest} = \text{Balance} \times \text{Interest Rate} / 100$$

$$\text{Total Balance} = \text{Balance} + \text{Interest}$$

5. Create an object of PremiumSavingsAccount and demonstrate:
 - Access to inherited attributes.
 - Multilevel inheritance.
 - Constructor invocation from the child class to the parent classes.
 - Calculation of the total balance including interest.

Class Hierarchy



Inheritance chain:

Account → SavingsAccount → PremiumSavingsAccount

Constructor Invocation Chain

When the following object is created:
PremiumSavingsAccount account =
 new PremiumSavingsAccount(101, 10000.0, 5.0,
 "Free ATM Withdrawals");

The constructors are invoked in the following order:

PremiumSavingsAccount Constructor

↓

SavingsAccount Constructor

↓

Account Constructor

So, although the object is created for PremiumSavingsAccount, the constructors of its parent classes are executed first.

Procedure

1. Define the Account class with accountNumber and balance.
2. Create a constructor in Account to initialize these attributes.
3. Define the SavingsAccount class by extending Account.
4. Add interestRate and initialize it using the constructor.
5. Define PremiumSavingsAccount by extending SavingsAccount.
6. Add extraBenefits and initialize all attributes using the constructor.
7. Use super() to invoke the constructors of the parent classes.
8. Implement calculateTotalBalance() in PremiumSavingsAccount.
9. Create an object of PremiumSavingsAccount.
10. Display the account details, calculated total balance, and additional benefits.

Expected Output

===== PREMIUM SAVINGS ACCOUNT =====

Account No: 101

Balance: 10000.0

Interest Rate: 5.0%

Total Balance (with interest): 10500.0

Benefits: Free ATM Withdrawals

Experiment6.3 : Hierarchical Inheritance and Runtime Polymorphism Using an E-Commerce Application

Aim: To design and implement an E-Commerce Application using hierarchical inheritance and demonstrate runtime polymorphism through method overriding and base-class references.

Problem Statement

Develop an E-Commerce Application with the following class hierarchy:

1. Create a base class Product with the following attributes:
 - id – unique product ID
 - name – name of the product
 - price – price of the product

Implement:

- A constructor to initialize the product details.
 - A method showDetails() to display the product information.
2. Create a subclass Clothing that inherits from Product and adds:
 - size – size of the clothing item.
 - material – material used.

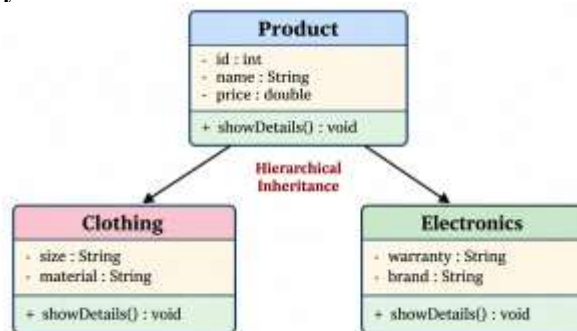
Override the showDetails() method to display the complete clothing details.

- warranty – warranty period.
 - brand – brand name.

Override the showDetails() method to display the complete electronics details.

4. Create an array of type Product and store objects of both Clothing and Electronics in it.
5. Demonstrate **runtime polymorphism** by invoking showDetails() using Product references.

Class Hierarchy



This is called hierarchical inheritance because multiple subclasses inherit from the same superclass.

Procedure

1. Define the Product superclass.
2. Declare id, name, and price.
3. Create a constructor to initialize the product details.
4. Define the showDetails() method in Product.
5. Create the Clothing subclass using extends Product.
6. Add size and material to Clothing.
7. Override showDetails() in Clothing.
8. Create the Electronics subclass using extends Product.
9. Add warranty and brand to Electronics.
10. Override showDetails() in Electronics.
11. Create a Product[] array.
12. Store Clothing and Electronics objects in the array.
13. Traverse the array and invoke showDetails().
14. Observe that the appropriate subclass method is executed at runtime.

Expected Output

===== PRODUCT DETAILS =====

Product ID: 201
Name: T-Shirt
Price: 499.0
Size: L
Material: Cotton

Product ID: 301
Name: Laptop
Price: 55000.0
Warranty: 2 years
Brand: Dell

4. Hybrid Inheritance (via Interfaces in Java)

(Java doesn't support multiple inheritance with classes, but interfaces allow hybrid structures.)

Scenario: You are developing a **Smart Device System**.

- Base class: Device (id, brand).
- Interface: WiFiEnabled (connectWiFi()).
- Interface: BluetoothEnabled (connectBluetooth()).
- Derived class: SmartPhone inherits from Device and implements both interfaces.

Question:

Implement this design. Show how SmartPhone object can use methods from both Device (class) and the two interfaces. Also explain why Java uses interfaces instead of multiple class inheritance.

Expected output:

Device ID: 101, Brand: Samsung
Connecting to WiFi...
Connecting to Bluetooth...

5. Multiple Inheritance (through Interfaces)

In a Payment System, implement the interfaces UPIPayment and CardPayment, and demonstrate how an object of the PaymentProcessor class can handle both UPI and Card payments.

- Interface UPIPayment has method upiPay(amount).
- Interface CardPayment has method cardPay(amount).
- Class PaymentProcessor implements both.

Expected output:

Paid 500.0 via UPI.
Paid 1500.0 via Card.